JWT 驗證練習教學

本教學將引導您如何使用 Spring Boot 和 JavaScript 實作 JWT (JSON Web Token) 驗證機制。

📋 目錄

1. 後端實作 (Java/Spring Boot)
    - JwtUtility 工具類別
    - UserController 控制器
2. 前端實作 (JavaScript/jQuery)
    - 登入功能
    - 驗證 Token 功能
3. 整合測試

---

1. 後端實作 (Java/Spring Boot)

pom.xml

```
<dependency>
    <groupId>javax.xml.bind</groupId>
    <artifactId>jaxb-api</artifactId>
    <version>2.3.1</version>
    <!-- javax.xml.bind.DatatypeConverter 解決找不到 -->
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
```

1.1 JwtUtility 工具類別

這個類別負責生成和驗證 JWT Token。

```
import java.util.*;
import org.springframework.stereotype.Component;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

@Component
public class JwtUtility {
    private static final String SECRET = "MySecretKey"; // 密鑰，實際專案應存放在配
置檔中

    // 生成 Token
    public static String generateToken(String username) {
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis() + 86400000)) //
有效期 1 天
                .signWith(SignatureAlgorithm.HS512, SECRET)
                .compact();
    }

    // 驗證 Token
    public static boolean validateToken(String token) {
        try {
            String name = Jwts.parser()
                    .setSigningKey(SECRET)
                    .parseClaimsJws(token)
                    .getBody()
                    .getSubject();
            return name != null;
        } catch (Exception e) {
            System.out.println("validateToken error " + e.getMessage());
            return false;
        }
    }

    // 從 Token 提取使用者名稱
    public String extractUsername(String token) {
        return Jwts.parser()
                .setSigningKey(SECRET)
                .parseClaimsJws(token)
                .getBody()
                .getSubject();
    }
}
```

1.2 UserController 控制器

處理使用者的登入請求。

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.ModelAndView;
import jakarta.servlet.http.HttpSession;
import java.util.*;

@RestController
@RequestMapping("/api/user")
@CrossOrigin // 允許跨來源請求
public class UserController {
    List<Map<String, String>> users = new ArrayList<>();

    @Autowired
    private JwtUtility jwtUtil;

    public UserController() {
        // 模擬使用者資料庫
        users.add(Map.of("admin", "1234"));
        users.add(Map.of("guest", "1234"));
        users.add(Map.of("mary", "1234"));
        users.add(Map.of("george", "1234"));
        users.add(Map.of("john", "1234"));
    }

    @GetMapping("/login")
    public ModelAndView showLogin() {
        return new ModelAndView("usercart");
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> payload,
HttpSession session) {
        String username = payload.get("username");
        String password = payload.get("password");

        // 驗證使用者帳密
        Map<String, String> user = users.stream()
            .filter(m -> password.equals(m.get(username)))
            .findAny()
            .orElse(null);

        if (user != null) {
            session.setAttribute("loginname", username);
            String token = jwtUtil.generateToken(username);
            System.out.println(username + " 登入成功");
            return ResponseEntity.ok(Map.of("token", token));
        } else {
```

```
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                                    .body(Map.of("message", "帳號或密碼錯誤"));
        }
    }

    // 新增: 驗證 Token 的 API
    @PostMapping("/validate")
    public ResponseEntity<?> validateToken(@RequestBody Map<String, String>
payload) {
        String token = payload.get("token");
        if (token != null && JwtUtility.validateToken(token)) {
            return ResponseEntity.ok(Map.of("valid", true, "message", "Token 有
效"));
        } else {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                                    .body(Map.of("valid", false, "message", "Token 無
效或已過期"));
        }
    }

    // 新增: 受保護的資源 API
    @GetMapping("/protected/resource")
    public ResponseEntity<?> getProtectedResource(@RequestHeader("Authorization")
String authHeader) {
        // 1. 檢查 Authorization Header 是否存在且格式正確
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Missing or
invalid Authorization header");
        }

        // 2. 提取 Token
        String token = authHeader.substring(7); // 去掉 "Bearer " 前綴

        // 3. 驗證 Token
        if (JwtUtility.validateToken(token)) {
            // Token 有效, 回傳受保護的資料
            String username = jwtUtil.extractUsername(token);
            return ResponseEntity.ok(Map.of(
                "message", "這是受保護的資料",
                "user", username,
                "timestamp", new Date()
            ));
        } else {
            // Token 無效
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid or
expired token");
        }
    }
}
```

---

2. 前端實作 (JavaScript/jQuery)

2.1 登入功能

當使用者點擊登入按鈕時，發送 AJAX 請求到後端，並將回傳的 Token 儲存到 `localStorage`。

```
$('#loginBtn').click(function (e) {
    e.preventDefault();
    const user = $('#username').val();
    const pass = $('#password').val();

    $.ajax({
        url: "http://localhost:8080/api/user/login",
        type: "POST",
        contentType: "application/json",
        data: JSON.stringify({ username: user, password: pass }),
        success: function (res) {
            // 儲存 Token 到 localStorage
            localStorage.setItem("token", res.token);
            console.log("Token stored:", res.token);

            isLoggedIn = true;
            $('#loginMessage').text('');
            $('#loginStatus').text(`歡迎, ${user}`);
            sessionStorage.setItem("username", `${user}`);
            alert("登入成功! ");

            // 登入成功後可以執行其他操作，例如跳轉頁面或載入受保護的資料
        },
        error: function (xhr) {
            $('#loginMessage').text('帳號或密碼錯誤');
            console.error("Login failed:", xhr);
        }
    });
});
```

2.2 驗證 Token 功能

在頁面載入時或執行敏感操作前，檢查 `localStorage` 中的 Token 是否有效。

```
// 檢查使用者是否已登入 (驗證 Token)
function checkLoginStatus() {
    const token = localStorage.getItem("token");

    if (!token) {
        console.log("No token found, user is not logged in.");
        handleLogout(); // 處理未登入狀態
        return;
    }

    // 這裡可以選擇發送請求到後端驗證 Token
    $.ajax({
```

```
        url: "http://localhost:8080/api/user/validate", // 假設後端有這個驗證 API
        type: "POST",
        contentType: "application/json",
        data: JSON.stringify({ token: token }),
        success: function(res) {
            if (res.valid) {
                console.log("Token is valid.");
                const username = sessionStorage.getItem("username") || "User";
                $('#loginStatus').text(`歡迎回來, ${username}`);
                isLoggedIn = true;
            } else {
                console.log("Token is invalid.");
                handleLogout();
            }
        },
        error: function() {
            console.log("Error validating token.");
            handleLogout();
        }
    });
}

// 處理登出或 Token 無效的情況
function handleLogout() {
    localStorage.removeItem("token");
    sessionStorage.removeItem("username");
    isLoggedIn = false;
    $('#loginStatus').text('請登入');
    // 可以在這裡顯示登入表單或重導向
}

// 頁面載入時執行檢查
$(document).ready(function() {
    checkLoginStatus();
});
```

2.3 發送帶有 Token 的請求

存取受保護的 API 資源時，需要在 Header 中帶上 Token。

```
function fetchProtectedData() {
    const token = localStorage.getItem("token");

    $.ajax({
        url: "http://localhost:8080/api/protected/resource",
        type: "GET",
        headers: {
            "Authorization": "Bearer " + token // 將 Token 加入 Authorization
Header
        },
        success: function(data) {
```

```
            console.log("Data received:", data);
        },
        error: function(xhr) {
            if (xhr.status === 401) {
                alert("登入已過期，請重新登入");
                handleLogout();
            }
        }
    });
}
```

---

3. 整合測試

1. **啟動後端服務**：確保 Spring Boot 應用程式已啟動並監聽 8080 port。
2. **開啟前端頁面**：在瀏覽器中開啟包含上述 JavaScript 的 HTML 頁面。
3. **測試登入**：
    - 輸入正確的帳號密碼 (例如: `admin` / `1234`)。
    - 點擊登入，確認 `localStorage` 中是否有 `token`。
    - 確認頁面顯示「歡迎，admin」。
4. **測試 Token 驗證**：
    - 重新整理頁面，確認是否自動保持登入狀態 (透過 `checkLoginStatus` 函式)。
    - 手動清除 `localStorage` 中的 token，重新整理頁面，確認是否變為未登入狀態。